Local Semantic File Finder — Project Checklist

A fully offline, cross-platform desktop app that lets users find personal files (PDFs, docs, images) by *meaning*, not filename — no cloud, no API calls, no LLM at runtime. Just OCR + embeddings + vector search.

Phase 1 — Foundation

1. Project setup

- [ ] Init repo, folder structure (`/core`, `/ui`, `/scripts`)
- [ ] Python env for core logic
- [ ] Install: `chromadb`, `ollama`, `pytesseract`, `pypdf`, `python-docx`, `watchdog`, `platformdirs`
- [ ] Install Tesseract binary (OS-specific) + Ollama + pull embedding model (`all-minilm`)

2. Decide vector DB

- [ ] Use **Chroma** (embedded, zero-server, ideal for single-user local desktop use)
- [ ] Decide storage path using `platformdirs` per OS
  - Windows: `%APPDATA%\FindIt\`
  - macOS: `~/Library/Application Support/FindIt/`
  - Linux: `~/.local/share/FindIt/`

3. Local service layer (not a public backend)

- [ ] Skip "FastAPI backend" framing — this is single-user, fully local
- [ ] Optionally run FastAPI bound to `127.0.0.1` only, purely as internal IPC bridge between Tauri frontend and Python logic (never exposed externally)
- [ ] Tauri frontend → localhost call → Python function → response

4. Choose OCR engine

- [ ] Use **Tesseract** via `pytesseract` (free, offline, cross-platform)
- [ ] Test early against real scanned docs (rotated images, low quality scans, photos of ID cards) to learn its real limits before building around it

---

Phase 2 — Ingestion Pipeline

5. File discovery

- [ ] Native OS folder-picker / permission dialog on first launch
- [ ] Recursive folder walk through chosen folders only

- [ ] Build full file list before processing anything

6. File type filtering (whitelist approach)

- [ ] Only process: `.pdf, .docx, .txt, .md, .jpg, .jpeg, .png`
- [ ] Skip: code files, system files, hidden folders (`.git`, `node_modules`, etc.)
- [ ] Skip oversized files by default (e.g. PDFs > 50MB), make configurable

7. Chunk + embed + store

- [ ] Extract text:
  - Native extraction for PDFs/docs first (`pypdf`, `python-docx`)
  - OCR fallback only if native extraction returns near-empty text
- [ ] Chunk at 500–1000 words
- [ ] Embed each chunk locally (`all-minilm` via Ollama)
- [ ] Store in Chroma with metadata: `{path, content_hash, filename, chunk_index, modified_time}`
- [ ] **Note:** don't try to cross-link identical chunks across files at ingestion time — that's unnecessary complexity for a personal vault. Exact-duplicate file detection (via whole-file hash) is a simple standalone v2 feature instead.

Phase 3 — Search

8. Hybrid search + ranking

- [ ] Strip noise words from query ("where is my", "find my", etc.)
- [ ] Embed cleaned query, cosine similarity search in Chroma (top ~10 candidates)
- [ ] Compute keyword/filename overlap score on same candidates
- [ ] Weighted combine (e.g. 75% semantic + 25% keyword)
- [ ] Dedup by file path (collapse multi-chunk matches from same file, keep highest score)
- [ ] Apply similarity threshold — exclude anything below cutoff entirely
- [ ] Return ranked list with similarity % for each result

9. Result UI behavior

- [ ] Dropdown-style result list below search bar, sorted by score
- [ ] Show filename + folder path + similarity % (+ optional text snippet)
- [ ] Click result → OS opens file natively
- [ ] Clear "No matches found" state when nothing clears threshold

---

Phase 4 — File Integrity Tracking

10. Live file watching

- [ ] `watchdog` observer on all indexed folders
- [ ] `on_moved` → update path in metadata (no re-embedding needed)
- [ ] `on_deleted` → mark entry as missing
- [ ] `on_created` → auto-ingest new file in background thread

11. Startup reconciliation

- [ ] On app launch, check all "missing" / path-invalid entries
- [ ] Re-hash current files in watched folders, match by content hash, relink if found
- [ ] If truly not found anywhere, keep marked missing (don't silently drop it)

Phase 5 — App Shell

12. Tauri setup

- [ ] Scaffold Tauri project
- [ ] Wire frontend (HTML/JS) to local Python backend via localhost calls
- [ ] Build single-search-bar UI (minimal, "just a search bar" look)

13. UI polish

- [ ] First-run consent + folder-picker screen
- [ ] Indexing progress bar ("340/1000 files indexed")
- [ ] Settings screen: manage indexed folders, re-scan button, clear index/reset

14. Packaging

- [ ] Build installers: Windows (.msi/.exe), macOS (.dmg, signed + notarized), Linux (.AppImage/.deb)
- [ ] Code signing (Apple Developer account, Windows code signing cert)
- [ ] Test permission prompts fresh on a clean machine per OS

Suggested Build Priority

Build and validate **Phase 2 + Phase 3** (steps 5–9) first as plain Python scripts, no UI at all. Prove search is fast and accurate against your own real, messy folder before touching Tauri or packaging. That's the riskiest and most important part — everything else (file tracking, UI, packaging) is comparatively mechanical once the search engine itself is proven.

Explicitly Out of Scope (for now)

- Cloud sync / webapp
- Any LLM/Claude API calls at runtime
- Google Drive or other cloud storage integration
- Duplicate file removal (planned as a future feature, not core v1)